



UNIVERSIDADE FEDERAL DO AMAPÁ
CURSO DE CIÊNCIA DA COMPUTAÇÃO
DEPARTAMENTO DE CIÊNCIAS EXATAS E TECNOLÓGICAS (DCET)

AYMMÉE NADINE DINIZ NERIS
EVELYN FIGUEIREDO DOS SANTOS
JOHNATHAN ROCHA DE SOUSA

RELATÓRIO TÉCNICO: TUTORIA - CC0121 INTELIGÊNCIA ARTIFICIAL

MACAPÁ AP
2026

1. Problema e Motivação

O ecossistema acadêmico do curso de Ciência da Computação da UNIFAP, especificamente na disciplina de Inteligência Artificial (CC0121), impõe uma alta carga cognitiva aos discentes. A assimilação de conceitos altamente abstratos (algoritmos de busca heurística, espaços de estados, aprendizado de máquina, agrupamento e redes neurais) ocorre em uma janela temporal reduzida de um semestre letivo.

1.1. Público Afetado

- A Dor: Alunos sofrem com a falta de suporte extraclasse personalizado e contextualizado. Ao recorrerem a LLMs comerciais genéricas (como ChatGPT ou Claude em suas versões públicas), os estudantes enfrentam dois problemas críticos: alucinação de conceitos (gerando definições que misturam frameworks que não foram adotados pelo professor) e desvio de escopo pedagógico (respostas complexas demais ou simplistas demais, desalinhadas com o plano de ensino da UNIFAP).
- Quem Sofre: O corpo discente da disciplina CC0121, que perde eficiência no estudo autônomo, e o docente, que precisa corrigir falhas conceituais originadas por respostas alucinadas de IAs comerciais.

1.2. Por que IA é a Resposta Certa (e não um Sistema If/Else)?

Uma abordagem de engenharia de software tradicional baseada em regras estáticas (**if/else**) ou correspondência de strings (string matching) é matematicamente inviável para este problema devido à explosão combinatória da linguagem natural.

Se um aluno pergunta "Como funciona o A", "Explique o algoritmo A estrela" ou "Qual heurística o A-star usa?", um sistema de regras precisaria prever cada variação sintática.

A IA baseada em Embeddings Semânticos resolve isso projetando as frases em um espaço vetorial latente multidimensional. Ela computa a proximidade por significado (geometria vetorial), independentemente da sintaxe. O pipeline RAG (Retrieval-Augmented Generation) "codado na unha" que desenvolvemos garante

que a flexibilidade da linguagem natural seja mantida no frontend, enquanto o conteúdo final entregue ao aluno fica matematicamente restrito aos limites dos documentos oficiais fornecidos pelo professor.

2. Dados: Ingestão, Volume e Análise Exploratória (EDA Textual)

2.1. Fonte e Volumetria dos Dados

Ao contrário de problemas de IA tradicionais baseados em datasets tabulares, o banco de dados foi construído a partir de dados textuais não estruturados proprietários. A fonte primária de dados consiste nos 8 módulos didáticos oficiais da disciplina, disponibilizados no formato HTML.

O processo de Análise Exploratória de Dados (EDA) focou na distribuição volumétrica do texto para identificar gargalos de densidade de informação por aula. Após a limpeza e o fatiamento (chunking), mapeamos o seguinte cenário de distribuição de dados:

2.2. Pré-processamento e Engenharia de Dados ([ingest_openai.py](#))

Para garantir a qualidade da recuperação e evitar o fenômeno de "vetores órfãos" ou "zeros impossíveis" (regiões do espaço vetorial corrompidas por caracteres de controle que proporcionaram a busca por cosseno), o script de ingestão executou o seguinte pipeline:

1. Limpeza de Tags e Scripts: Extração do texto puro contido nos HTMLs utilizando padrões Regex e exclusão de tags de estilização, metadados de cabeçalho e scripts de renderização que não continham valor pedagógico.
2. Estratégia de Chunking Dinâmico: O texto de cada aula foi fragmentado em blocos textuais (chunks) otimizados. Definimos um teto de tokens que varia entre 500 e 1000 por bloco. Essa escolha mitiga dois problemas: blocos pequenos demais perdem o contexto necessário para a LLM responder; blocos grandes demais diluem o sinal do embedding, reduzindo a acurácia da busca vetorial.

3. Metodologia: Arquitetura e Engenharia de IA

A arquitetura do TutorIA CC0121 foi projetada com base no desacoplamento estrito entre a camada de interface (Next.js 16) e a engine de IA (FastAPI).

3.1. Engenharia do Pipeline de RAG (Fluxo Online)

Ao receber uma requisição no endpoint `POST /api/v1/chat`, a engenharia do backend executa os seguintes passos coordenados:

1. Vetorização em Tempo Real: A pergunta textual do aluno é repassada ao `embedding_service`, acionando o modelo `text-embedding-3-small`. Configuramos a API para retornar uma representação vetorial compacta e densa de 512 dimensões, reduzindo o custo de armazenamento e a latência de busca sem perda perceptível de granularidade semântica.
2. Recuperação Semântica por Similaridade de Cosseno: O vetor gerado é submetido ao índice do Pinecone via `pinecone_service`. A busca realiza o cálculo da distância angular entre o vetor da pergunta (A) e os vetores dos blocos textuais das aulas (B) armazenados. O banco retorna os Top-K = 5 blocos de texto com maior pontuação de similaridade.
3. Montagem do Prompt Injetado (`prompt_builder.py`): O backend extrai o texto cru desses 5 blocos e os anexa a uma estrutura de prompt rígida de sistema.
4. Inferência e Geração: O prompt estruturado é enviado ao modelo `gpt-4o-mini`. Configuramos o hiperparâmetro de temperatura em 0.2. Próxima de zero, a amostragem de tokens torna-se altamente determinística, forçando o modelo a se comportar estritamente como um sumariizador do contexto injetado, mitigando alucinações.

3.3. Mecanismo Adaptativo: Por que o Plano de Estudos Não Usa LLM?

Uma decisão crítica de design de software foi tomada no desenvolvimento do módulo `/api/v1/study-plan`. Inicialmente, cogitou-se passar o resultado dos erros do quiz de diagnóstico para a LLM gerar o plano de estudos via Structured Outputs. No entanto, durante a fase de testes, essa abordagem demonstrou-se ineficiente, gerando latência desnecessária e risco de quebra de linearidade pedagógica.

A Solução Final Engenhada: Desenvolvemos o `local_study_plan_service`, um motor 100% determinístico rodando localmente no backend. Quando o aluno submete o quiz de diagnóstico (`/api/v1/diagnostic/submit`), o serviço avalia as respostas com base em uma matriz de gabarito estática e gera uma lista de strings contendo as fraquezas identificadas.

O motor então mapeia essas fraquezas diretamente para passos de estudo pré-configurados e estruturados em um grafo de dependências. O plano de estudos final é gerado instantaneamente (latência de microssegundos) com referências exatas de aulas e tópicos sequenciais, sem gastar tokens e com estabilidade absoluta de schema (garantida via validação Pydantic).

4. Resultados: Métricas de Avaliação e Performance

Para validar o comportamento do ecossistema, o sistema foi submetido a testes de carga automatizados e testes de sanidade lógica baseados nos Requisitos Não Funcionais (RNFs) e Regras de Negócio (RNs) mapeados.

4.1. Tabela de Validação de Engenharia

Requisito / Regra	Métrica Alvo (SLA)	Desempenho Medido nos Testes	Status Técnico
RNF02 - Latência de Chat	Tempo de resposta total	Média estável de 3.2 segundos sob concorrência.	Aprovado
RN02 - Rastreabilidade de Fonte	100% das respostas com metadados reais	Retorno íntegro dos campos <code>aula</code> e <code>título</code> vinculados aos blocos.	Aprovado

RN03 - Blindagem de Escopo	Rejeição imediata de tópicos extra-curriculares	Bloqueio imediato via prompt-system com limite de corte.	Aprovado
CA06 - Estabilidade de Schema	Payload JSON válido no plano de estudos	Validação Pydantic com 0% de exceções de parsing.	Aprovado

4.2. Análise do Perfil de Latência (Gargalos de Infraestrutura)

Abaixo, detalhamos o mapeamento do tempo de execução de uma requisição padrão no endpoint `/api/v1/chat` para identificar os gargalos do pipeline.

Justificativa de Métricas: A latência de geração (2.2s) representa o maior peso do pipeline. Isso se deve à natureza autoregressiva do modelo `gpt-4o-mini` ao realizar a amostragem (sampling) token a token. O uso do FastAPI assíncrono (`async/await`) impede que esse gargalo bloqueie o loop de eventos do servidor, mantendo a API responsiva para múltiplos usuários simultâneos.

4.3. Filtro de Segurança e Limiar de Confiança

Para implementar a regra RN03 (Resposta Insuficiente), avaliamos o comportamento do Score de Similaridade de Cosseno retornado pelo Pinecone. Durante as rodadas de testes, estabelecemos um limiar de corte de 0.70.

Se o bloco com maior proximidade da pergunta do aluno retornar um score inferior a 0.70, o backend intercepta o fluxo, descarta o contexto e instrui a LLM a emitir a resposta padrão de segurança: "Desculpe, mas não encontrei informações suficientes nos materiais oficiais da disciplina para responder à sua pergunta de

forma confiável." Isso impede que o modelo tente inferir respostas usando seu conhecimento prévio de internet.

5. Análise de Riscos, Ética e Limitações Sistêmicas

Alinhados com a engenharia de IA responsável discutida na Aula 08, a ética foi integrada como requisito técnico de arquitetura.

5.1. Vieses de Dados e Métricas de Fairness (Auditoria de IA)

Modelos de aprendizado estatístico herdam vieses históricos contidos nos dados de treinamento. No contexto do TutorIA, mapeamos os seguintes riscos:

- **Viés de Linguagem e Representação:** O tokenizador do modelo `text-embedding-3-small` opera sob a arquitetura de tokens (fatiamento de strings em sub-palavras). Palavras raras, acentuadas ou termos coloquiais e gírias da comunidade acadêmica da UNIFAP e da região norte sofrem fragmentação excessiva, gerando uma maior contagem de tokens e penalizando de forma artificial o cálculo de similaridade semântica. Isso pode causar falsos alertas de fora de escopo (bloqueio por score < 0.70) para alunos que não utilizem a norma culta padrão idêntica aos slides.
- **Protocolo de Auditoria Técnica (Regra dos 4/5 ou 80%):** Conforme os padrões de auditoria de fairness, o time técnico deve monitorar a razão de impacto das aprovações/recusas de respostas entre subgrupos de usuários formulando perguntas formais vs. informais
Se a razão for inferior a 0.80, o sistema emite um alerta de discriminação algorítmica no pipeline RAG, exigindo reajuste no limiar do vetor ou enriquecimento do léxico.

5.2. Limitações Técnicas Operacionais

- **O que o Modelo NÃO Consegue Fazer:** O pipeline de ingestão atual ignora componentes visuais, tabelas complexas e diagramas contidos nos HTMLs originais. Ele é incapaz de processar informações multimodais, restringindo-se ao texto plano extraído.
- **Arquitetura Stateless:** A ausência de um banco de dados relacional persistente impede a retenção do histórico de chats e da evolução do desempenho dos quizzes do discente.

- **Regulação Legal:** O software foi arquitetado em conformidade com as diretrizes de privacidade da LGPD (exigência de explicação de decisões automatizadas de crédito de conhecimento) e inspirando-se nos frameworks de classificação de risco do AI Act europeu e do PL 2338 (Marco Legal da IA no Brasil).

6. Conclusão e Próximos Passos (Sprint Futura)

O MVP do TutorIA CC0121 consolidou com sucesso um pipeline de suporte educacional blindado e de baixa latência. Em uma sprint adicional de 30 dias de engenharia, o foco será direcionado para:

1. **Camada de Persistência Relacional:** Acoplamento do PostgreSQL via SQLAlchemy no FastAPI para registrar trajetórias de aprendizado e telemetria de falhas da turma.
2. **Soberania de Infraestrutura (RAG 100% Local):** Substituição das APIs proprietárias da OpenAI por modelos open-source executados localmente em servidores da universidade, eliminando custos e dependências externas.

REFERÊNCIAS

FASTAPI. FastAPI Framework Documentation: Production-ready ASGI applications. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 2026.

NEXT.JS. Next.js 16 Architecture and Routing Specifications. Disponível em: <https://nextjs.org/docs>. Acesso em: 2026.

OPENAI. OpenAI API Reference: Vector Embeddings & Structured Outputs. Disponível em: <https://platform.openai.com/docs/>. Acesso em: 2026.

PINECONE. Pinecone Vector Database: Architecture for Vector Similarity Search. Disponível em: <https://docs.pinecone.io/>. Acesso em: 2026.

UNIFAP. Aula 08 - Tópicos Modernos: LLMs, RAG, Agentes e Ética - Disciplina CC0121 Inteligência Artificial. Amapá, 2026.